GitHub Copilot in the CLI

Choose the surface that fits the work

Session 04 | 3 hours

Agenda

Segment	Time
When the CLI fits	10 min
Modes and control	15 min
Programmatic use	10 min
CLI and IDE together	10 min
Safety and evidence	10 min
Lab handoff	5 min

Start with the task

Choose the CLI when the work starts or ends in the terminal:

- reading logs or command output;
- working over SSH or in a container;
- running a repeatable command sequence;
- producing output for another script.

Use the IDE when visual navigation and diff review matter more.

Match the mode to the risk

Mode	Use it when
Standard	The task is small or still changing
Plan	You want to review the approach before edits
Autopilot	The task is bounded and recovery is clear

Permissions still apply. More autonomy requires a narrower task.

A bounded task is easier to trust

State:

- the working directory;
- the files or commands in scope;
- the expected result;
- the checks to run;
- when Copilot must stop.

Do not automate an unclear request.

Programmatic mode needs a contract

```
copilot -p "Summarize the failing test output"
```

A script needs more than a useful answer. Define:

- stable input;
- a predictable output format;
- a failure signal;
- limited permissions.

Free-form output is not a reliable pipeline interface.

CLI and IDE can share one workflow

```
CLI
  reproduce the failure
  capture focused evidence
        ↓
IDE
  inspect the code and diff
  review the proposed change
        ↓
CLI
  rerun checks
  record the result
```

Switch surfaces when the work changes.

Context is not authority

The CLI can see terminal state, files, Git state, and command output.

That context helps Copilot reason. It does not decide:

- which data is approved;
- which commands are safe;
- whether the change is correct;
- whether the result should ship.

A person owns those decisions.

Put gates around side effects

Before a command changes files or external systems:

1. confirm the directory and scope;
2. inspect the proposed action;
3. limit tools and credentials;
4. define a rollback or stop rule;
5. verify the result outside the conversation.

Use the smallest permission that completes the task.

Evidence should survive the session

Keep:

- the final diff;
- test output and exit status;
- generated reports;
- the prompt and stated limits;
- the final human decision.

Conversation history alone is not a delivery record.

Lab handoff

Use the CLI to inspect a project, repair a bounded defect, produce a health report, and compare the CLI and IDE workflows.

Success looks like: passing checks, a reviewed diff, and evidence another person can inspect.